Chapter 12: Linear Programming

Algorithms for Optimization, 2nd ed. (2026)

Kochenderfer & Wheeler

2026

Table of Contents

What is Linear Programming?

Definition

A **linear program (LP)** consists of a *linear objective function* and a set of *linear constraints*:

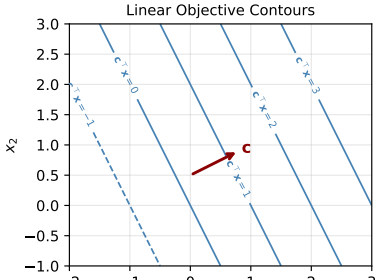
$$\underset{x}{\text{minimize}}\ c^\top x \quad \text{subject to linear equalities/inequalities in } x$$

Key Properties

- Linear objective $\Rightarrow$ flat "ramp"
- Feasible set is a *convex polytope*
- Any local minimum is a *global* minimum
- Modern solvers: millions of variables & constraints

Applications:

- Transportation & logistics
- Communication networks
- Manufacturing, economics
- Operations research
- L_1 / L_∞ regression (converted to LP)



General Form (Eq. 12.2)

$$\underset{x}{\text{minimize}}\ c^\top x \quad \text{subject to} \quad A_{\text{LE}}\, x \le b_{\text{LE}}, \quad A_{\text{EQ}}\, x = b_{\text{EQ}}$$

- $c, x \in \mathbb{R}^n$ — cost vector and design variables
- $A_{\text{LE}} \in \mathbb{R}^{p\times n}$, $b_{\text{LE}} \in \mathbb{R}^p$ — inequality rows
- $A_{\text{EQ}} \in \mathbb{R}^{q\times n}$, $b_{\text{EQ}} \in \mathbb{R}^q$ — equality rows

Example 12.1 — Concrete LP

$$\begin{aligned}\underset{x_1,x_2,x_3}{\text{minimize}}\ & 2x_1 - 3x_2 + 7x_3 \\ \text{s.t.}\ & 2x_1 + 3x_2 - 8x_3 \le 5 \\ & 4x_1 + x_2 + 3x_3 \le 9 \\ & x_1 - 5x_2 - 3x_3 \ge -4 \\ & x_1 + x_2 + 2x_3 = 1\end{aligned}$$

Example 12.2 — Norm Minimization as LP

minimize $\|Ax - b\|_1$ is equivalent to:

$$\underset{x,s}{\text{minimize}}\ 1^\top s \quad \text{s.t.} \quad Ax - b \le s,\ Ax - b \ge -s$$

minimize $\|Ax - b\|_\infty \Leftrightarrow$ minimize scalar t :

Standard Form

Standard Form (Eq. 12.3)

$$\underset{x}{\text{minimize}}\ c^\top x \quad \text{subject to} \quad Ax \le b, \quad x \ge 0$$

All constraints are *less-than inequalities*; all variables are *nonnegative*.

Conversion rules:

1. **Equality → two inequalities** (Eq. 12.4):

$$A_{EQ}x = b_{EQ} \longrightarrow A_{EQ}x \le b_{EQ}, \quad -A_{EQ}x \le -b_{EQ}$$

2. **Free variable → positive/negative split** (Eq. 12.6): Replace x with $x^+ - x^-$, constrain $x^+, x^- \ge 0$:

$$\underset{x^+,x^-}{\text{minimize}} \begin{bmatrix} c^\top & -c^\top \end{bmatrix} \begin{bmatrix} x^+ \\ x^- \end{bmatrix} \text{ s.t. } \begin{bmatrix} A & -A \end{bmatrix} \begin{bmatrix} x^+ \\ x^- \end{bmatrix} \le b, \ \begin{bmatrix} x^+ \\ x^- \end{bmatrix} \ge 0$$

Equality Form and Slack Variables

Equality Form (Eq. 12.7)

$$\underset{x}{\text{minimize}}\ c^\top x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0$$

- $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$
- m equality constraints, n nonneg. variables
- Preferred form for the simplex algorithm

Standard $\rightarrow$ Equality via Slack Variables (Eq. 12.8)

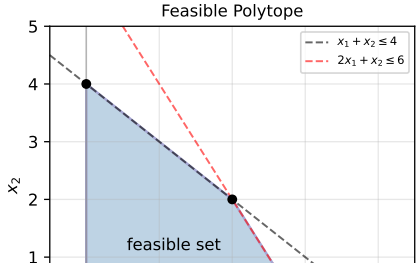
$$Ax \leq b \xrightarrow{\text{add } s \geq 0} Ax + s = b, \quad s \geq 0$$

Example 12.3 — Slack Variable Illustration

Standard form: $\text{minimize}_x\ x$ s.t. $x \geq 1$
Equality form (add slack $s \geq 0$):

$$\underset{x,s}{\text{minimize}}\ x \quad \text{s.t.} \quad x - s = 1,\ x, s \geq 0$$

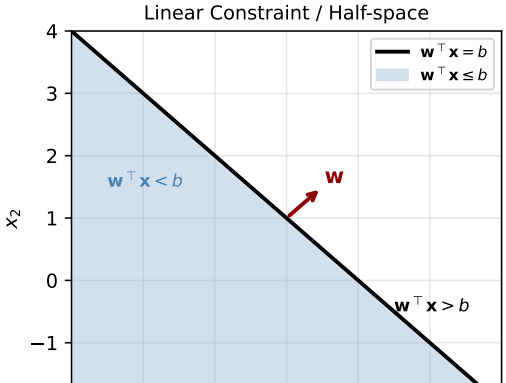
The equality constraint restricts feasible points to the line $x - s = 1$.



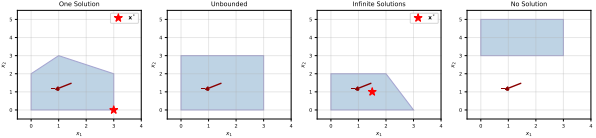
Geometric Interpretation

Half-space:

- Constraint $\mathrm{w}^\top \mathrm{x} \leq b$ defines a *half-space*
- Hyperplane $\mathrm{w}^\top \mathrm{x} = b$ is perpendicular to w
- Region $\mathrm{w}^\top \mathrm{x} > b$ is on the $+\mathrm{w}$ side
- Region $\mathrm{w}^\top \mathrm{x} < b$ is on the $-\mathrm{w}$ side



Four outcome types (Figure 12.4):



- **One solution**: optimal at a vertex
- **Unbounded**: feasible set extends to $-\infty$
- **Infinite solutions**: optimal face $\perp$ c
- **No solution**: over-constrained, infeasible

Example 12.4 — Standard to Equality Form

Conversion: maximize $5x_1 + 4x_2$ subject to $2x_1 + 3x_2 \leq 5$, $4x_1 + x_2 \leq 11$

Step 1: Convert to minimization (negate objective):

$$\underset{x_1, x_2}{\text{minimize}} \; -5x_1 - 4x_2$$

Step 2: Introduce slack variables $s_1, s_2 \geq 0$:

$$\begin{aligned} 2x_1 + 3x_2 + s_1 &= 5 \\ 4x_1 + \; x_2 + s_2 &= 11 \end{aligned}$$

Step 3: Split free variables x_1, x_2 into $x_i^+, x_i^- \geq 0$:

$$\underset{x_1^+, x_1^-, x_2^+, x_2^-, s_1, s_2}{\text{minimize}} \quad -5(x_1^+ - x_1^-) - 4(x_2^+ - x_2^-)$$

$$\text{s.t.} \quad 2(x_1^+ - x_1^-) + 3(x_2^+ - x_2^-) + s_1 = 5, \quad 4(x_1^+ - x_1^-) + (x_2^+ - x_2^-) + s_2 = 11$$

$$x_1^+, x_1^-, x_2^+, x_2^-, s_1, s_2 \geq 0$$

Python: Building LP Standard/Equality Form

```
import numpy as np
from scipy.optimize import linprog

# -- Example 12.1 via scipy linprog ----------------------------------------
# minimize  2x1 - 3x2 + 7x3
# s.t.      2x1 + 3x2 - 8x3 <=  5
#           4x1 +  x2 + 3x3 <=  9
#          -x1 + 5x2 + 3x3  <=  4    (flipped from x1-5x2-3x3 >= -4)
#           x1 +  x2 + 2x3  ==  1    (equality)

c      = np.array([ 2.0, -3.0,  7.0])
A_ub   = np.array([[ 2,  3, -8],
                   [ 4,  1,  3],
                   [-1,  5,  3]])
b_ub   = np.array([5.0, 9.0, 4.0])
A_eq   = np.array([[1, 1, 2]])
b_eq   = np.array([1.0])

res = linprog(c, A_ub=A_ub, b_ub=b_ub,
              A_eq=A_eq, b_eq=b_eq, method='highs')
print("Optimal x :", res.x)
print("Optimal f :", res.fun)

# -- L1 minimization as LP (Example 12.2) ----------------------------------
def l1_min_lp(A, b):
    """Minimize ||Ax - b||_1  via LP."""
    m, n = A.shape
```

12.2 Simplex Algorithm — Overview

Core Idea

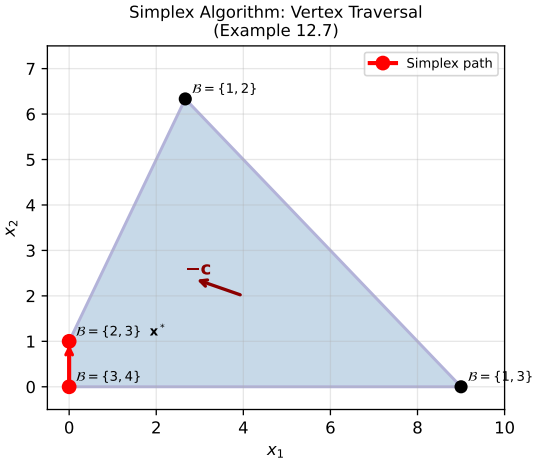
Move from **vertex to vertex** of the feasible polytope, decreasing the objective at each step, until optimality is confirmed.

Two phases:

1. **Initialization phase**: Find an initial feasible vertex (by solving an auxiliary LP)
2. **Optimization phase**: Traverse adjacent vertices following the steepest descent direction

Requirements for the Simplex Method

- LP must be in **equality form**: $Ax = b$, $x \geq 0$
- Rows of A must be **linearly independent**
- Number of constraints $m \leq n$ (not



Simplex path on feasible polytope (Example 12.7): $\mathcal{B} = \{3, 4\} \to \mathcal{B} = \{2, 3\}$ (optimal)

12.2.1 Vertices and Partitions

Vertex Characterization

Every vertex of the feasible polytope $\{x : Ax = b, x \geq 0\}$ is uniquely defined by setting exactly $n - m$ components of x to zero.
The remaining n indices are partitioned into:

$$\mathcal{B} \cup \mathcal{V} = \{1, \ldots, n\}, \quad |\mathcal{B}| = m, \ |\mathcal{V}| = n - m$$

- $\mathcal{B}$ ("busy" / basic): $i \in \mathcal{B} \Rightarrow x_i \geq 0$ (Eq. 12.12)
- $\mathcal{V}$ ("vacant" / non-basic): $i \in \mathcal{V} \Rightarrow x_i = 0$ (Eq. 12.11)

Extracting a Vertex (Eq. 12.13) — Algorithm 12.1

The $m \times m$ submatrix $A_{\mathcal{B}}$ formed by the m columns of A indexed by $\mathcal{B}$ must be *invertible*:

$$Ax = A_{\mathcal{B}} x_{\mathcal{B}} = b \quad \Rightarrow \quad x_{\mathcal{B}} = A_{\mathcal{B}}^{-1} b$$

Example 12.6 — Verify a Vertex

$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & -1 & 2 & 3 \\ 2 & 1 & 2 & -1 \end{bmatrix}$, $x = [1, 1, 0, 0]$

Try $\mathcal{B} = \{1, 2, 3\}$: $A_{\{1,2,3\}}$ is invertible ✓
Try $\mathcal{B} = \{1, 2, 4\}$: $A_{\{1,2,4\}}$ is invertible ✓

12.2.2 First-Order Necessary Conditions (KKT)

Lagrangian for Equality-Form LP (Eq. 12.14)

$$\mathcal{L}(x, \boldsymbol{\mu}, \boldsymbol{\lambda}) = c^\top x - \boldsymbol{\mu}^\top x - \boldsymbol{\lambda}^\top (Ax - b)$$

Four necessary (and sufficient for LPs) KKT conditions:

1. **Feasibility**: $Ax = b$, $x \geq 0$
2. **Dual feasibility**: $\boldsymbol{\mu} \geq 0$
3. **Complementary slackness**: $\boldsymbol{\mu} \odot x = 0$
4. **Stationarity**: $A^\top \boldsymbol{\lambda} + \boldsymbol{\mu} = c$

Computing $\boldsymbol{\lambda}$ and $\boldsymbol{\mu}_\mathcal{V}$ at a vertex:
Set $\boldsymbol{\mu}_\mathcal{B} = 0$ (complementary slackness with $x_\mathcal{B} \geq 0$):

$$A_\mathcal{B}^\top \boldsymbol{\lambda} = c_\mathcal{B} \quad \Rightarrow \quad \boldsymbol{\lambda} = A_\mathcal{B}^{-\top} c_\mathcal{B} \tag{12.17}$$

Then (Eq. 12.20):

$$\boldsymbol{\mu}_\mathcal{V} = c_\mathcal{V} - \left(A_\mathcal{B}^{-1} A_\mathcal{V}\right)^\top c_\mathcal{B}$$

Optimality Test

- If $\boldsymbol{\mu}_\mathcal{V} \geq 0$: **current vertex is optimal**
- If any $(\boldsymbol{\mu}_\mathcal{V})_q < 0$: vertex is suboptimal; pivot on index q

12.2.3 Optimization Phase — Pivoting

Edge Transition (Eq. 12.21–12.24)

Choose **entering index** $q \in \mathcal{V}$ (with $(\mu_{\mathcal{V}})_q < 0$). New vertex must satisfy $Ax' = b$:

$$x'_{\mathcal{B}} = x_{\mathcal{B}} - A_{\mathcal{B}}^{-1} A_{\{q\}}\, x'_q$$

Minimum ratio test — choose **leaving index** $p \in \mathcal{B}$:

$$x'_q = \min_{p \in \mathcal{B},\, d_p > 0} \frac{(x_{\mathcal{B}})_p}{d_p}, \quad \text{where } d = A_{\mathcal{B}}^{-1} A_{\{q\}}$$

Swap p out of $\mathcal{B}$, put q in: $\mathcal{B}' \leftarrow (\mathcal{B} \setminus \{p\}) \cup \{q\}$.

Objective Change (Eq. 12.32)

$$c^\top x' - c^\top x = \mu_q\, x'_q$$

Decreases only if $\mu_q < 0$ *and* $x'_q > 0$.

Entering Index Heuristics

- **Greedy**: choose q that *maximally* reduces $c^\top x$
- **Dantzig's rule**: choose q with most negative μ_q
  (*easy to compute, scaling-sensitive*)
- **Bland's rule**: choose the *first* q with $\mu_q < 0$
  (*prevents cycling, slower in practice*)

Cycling

If no improvement is made over several iterations, Bland's rule breaks cycles and guarantees convergence.

Algorithm 12.3 & 12.4 (Python) — Simplex Step

```python
import numpy as np

def get_vertex(B, A, b):
    """Algorithm 12.1: Extract vertex from partition B."""
    b_inds = sorted(B)
    AB = A[:, b_inds]
    xB = np.linalg.solve(AB, b)
    x = np.zeros(A.shape[1])
    x[b_inds] = xB
    return x

def edge_transition(A, B, q):
    """Algorithm 12.2: Compute leaving index p and new x'_q."""
    n = A.shape[1]
    b_inds = sorted(B)
    n_inds = sorted(set(range(n)) - set(b_inds))
    AB = A[:, b_inds]
    xB = np.linalg.solve(AB, A[:, n_inds[q]])  # d = A_B^{-1} A_{q}
    # xB_current needed: pass separately in full implementation
    return xB   # direction d

def step_lp(B, A, b, c):
    """Algorithm 12.3: One simplex step (greedy heuristic)."""
    n = A.shape[1]
    b_inds = sorted(B)
    n_inds = sorted(set(range(n)) - set(b_inds))
    AB, AV = A[:, b_inds], A[:, n_inds]
```

Example 12.7 — Simplex in Action

Setup

$$A = \begin{bmatrix} 1 & 1 & 1 & 0 \\ -4 & 2 & 0 & 1 \end{bmatrix},\ b = \begin{bmatrix} 9 \\ 2 \end{bmatrix},\ c = \begin{bmatrix} 3 \\ -1 \\ 0 \\ 0 \end{bmatrix}$$

Initial partition: $\mathcal{B} = \{3, 4\}$ (slacks)

Iteration 1

$$x_{\mathcal{B}} = A_{\mathcal{B}}^{-1} b = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 9 \\ 2 \end{bmatrix} = \begin{bmatrix} 9 \\ 2 \end{bmatrix}$$

$$\boldsymbol{\lambda} = A_{\mathcal{B}}^{-\top} c_{\mathcal{B}} = 0, \quad \boldsymbol{\mu}_{\mathcal{V}} = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

$\boldsymbol{\mu}_{\mathcal{V}}$ has negative entry: $q = 2$ enters.
Min ratio $\Rightarrow$ $p = 4$ leaves.
Update: $\mathcal{B} \leftarrow \{2, 3\}$.

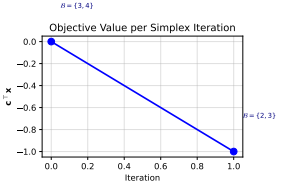
Iteration 2 (optimal)

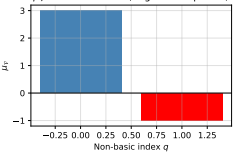
$$x_{\mathcal{B}} = \begin{bmatrix} 1 \\ 8 \end{bmatrix}, \quad \boldsymbol{\lambda} = \begin{bmatrix} 0 \\ -1/2 \end{bmatrix}$$

$$\boldsymbol{\mu}_{\mathcal{V}} = \begin{bmatrix} 1 \\ 1/2 \end{bmatrix} \geq 0 \ \checkmark$$

Optimal: $\mathcal{B} = \{2, 3\}$, $x^* = [0, 1, 8, 0]^\top$

$$c^\top x^* = 3 \cdot 0 + (-1) \cdot 1 = -1$$





Objective decreases and $\mu_{\mathcal{V}}$ at initial vertex

The Simplex Tableau

Tableau Representation

At any vertex with partition $(\mathcal{B}, \mathcal{V})$, the **simplex tableau** compactly encodes all quantities needed for pivoting:

	$x_{\mathcal{B}}$	$x_{\mathcal{V}}$
$A_{\mathcal{B}}^{-1}A$	I	$A_{\mathcal{B}}^{-1}A_{\mathcal{V}}$
$c^\top - c_{\mathcal{B}}^\top A_{\mathcal{B}}^{-1}A$	0	$\boldsymbol{\mu}_{\mathcal{V}}^\top$

- **Right-hand side**: $x_{\mathcal{B}} = A_{\mathcal{B}}^{-1}b$
- **Reduced costs**: $\boldsymbol{\mu}_{\mathcal{V}}$ (bottom row)
- Pivot = row operations to swap columns

Minimum Ratio Test Illustrated

To introduce q into $\mathcal{B}$, the direction of travel in $x_{\mathcal{B}}$-space is $d = A_{\mathcal{B}}^{-1}A_{\{q\}}$.
We increase x_q until the first basic variable hits zero:

$$x_q' = \min_{i:\, d_i > 0} \frac{(x_{\mathcal{B}})_i}{d_i}$$

- If all $d_i \leq 0$: LP is **unbounded**
- Ties (degeneracy): Bland's rule prevents infinite cycling

12.2.4 Initialization Phase — Auxiliary LP

Problem: How to Find an Initial Feasible Vertex?

Introduce auxiliary variables $z \in \mathbb{R}^m$ (Eq. 12.33):

$$\underset{x,z}{\text{minimize}} \begin{bmatrix} 0^\top & 1^\top \end{bmatrix} \begin{bmatrix} x \\ z \end{bmatrix} \quad \text{s.t.} \quad \begin{bmatrix} A & Z \end{bmatrix} \begin{bmatrix} x \\ z \end{bmatrix} = b, \quad \begin{bmatrix} x \\ z \end{bmatrix} \geq 0$$

where $Z_{ii} = +1$ if $b_i \geq 0$, -1 otherwise.

Procedure:

1. Start with $x = 0$, $z_j = |b_j|$ (initial basic partition = z-columns)
2. Solve auxiliary LP via optimization phase
3. If optimal $z^* = 0$: feasible vertex found
4. If $z^* \neq 0$: original LP is **infeasible**
5. Replace z-entries in $\mathcal{B}$ with x-entries as needed

Example 12.8 — Finding Feasible Vertex

LP: $2x_1 - x_2 + 2x_3 = 1$, $5x_1 + x_2 - 3x_3 = -2$
Auxiliary: add $z_1, z_2 \geq 0$:

$$2x_1 - x_2 + 2x_3 + z_1 = 1, \quad 5x_1 + x_2 - 3x_3 - z_2 = -2$$

Init: $\mathcal{B} = \{4, 5\}$, $x^{(1)} = [0, 0, 0, 1, 2]$
After solving: $x^* \approx [0, 1, 1, 0, 0]$, $\mathcal{B} = \{2, 3\}$
Feasible vertex: $[0, 1, 1]$ ✓

Algorithm 12.5 (Python) — Complete Simplex

```python
import numpy as np

def find_partition(A, b, c):
    """Algorithm 12.5: Find initial feasible vertex via auxiliary LP."""
    m, n = A.shape
    # Build Z: diagonal with +1 if b_i>=0 else -1
    Z = np.diag([1.0 if bi >= 0 else -1.0 for bi in b])
    A_aux = np.hstack([A, Z])
    b_aux = b.copy()
    c_aux = np.concatenate([np.zeros(n), np.ones(m)])
    # Initial partition: last m columns (z-variables), 0-indexed
    B = set(range(n, n + m))
    B, _ = minimize_lp(B, A_aux, b_aux, c_aux)
    # Replace z-indices in B with unused x-indices
    available_xs = sorted(set(range(n)) - B)
    zs_to_replace = sorted([j for j in B if j >= n])
    for k, j in enumerate(zs_to_replace):
        if k < len(available_xs):
            B = (B - {j}) | {available_xs[k]}
    return B

def minimize_lp(B, A, b, c):
    """Algorithm 12.4: Minimize LP from initial partition B."""
    done = False
    while not done:
        B, done = step_lp(B, A, b, c)
    x = get_vertex(B, A, b)
```

Python: Solving an LP with SciPy (Practical Usage)

```python
from scipy.optimize import linprog
import numpy as np

# -- Example 12.7 via scipy (standard form) ---------------------------------
# LP: min  3x1 - x2   s.t. x1+x2<=9, -4x1+2x2<=2, x1,x2>=0
c      = np.array([3.0, -1.0])
A_ub   = np.array([[ 1.0,  1.0],
                   [-4.0,  2.0]])
b_ub   = np.array([9.0, 2.0])
bounds = [(0, None), (0, None)]     # x1, x2 >= 0

result = linprog(c, A_ub=A_ub, b_ub=b_ub,
                 bounds=bounds, method='highs')
print("Optimal x  :", result.x)
print("Optimal obj:", round(result.fun, 4))
# Optimal x  : [0. 1.]
# Optimal obj: -1.0

# -- Dual variables (Lagrange multipliers) ----------------------------------
# HiGHS method returns dual values via result.ineqlin.marginals
print("Dual (lambda):", result.ineqlin.marginals)

# -- Example: L-infinity minimization ---------------------------------------
def linf_regression(A, b):
    """Solve min_x ||Ax - b||_inf  as an LP."""
    m, n = A.shape
    # Variables: [x (n), t (1)].  Minimize t.
```

Simplex Complexity & Summary

Complexity

- Each iteration: $O(m^2 n)$ for basis update
- Worst case: exponential (Klee-Minty examples)
- **Average case**: polynomial in practice
- Modern implementations (HiGHS, CPLEX, Gurobi)
  solve millions of variables & constraints

Variants

- **Revised simplex**: updates $\mathrm{A}_{\mathcal{B}}^{-1}$ incrementally
- **Interior-point methods**: polynomial worst case
- **Self-dual simplex**: avoids two-phase

Step Summary: One Simplex Iteration

1. Compute $\mathrm{x}_{\mathcal{B}} = \mathrm{A}_{\mathcal{B}}^{-1}\mathrm{b}$
2. Compute $\boldsymbol{\lambda} = \mathrm{A}_{\mathcal{B}}^{-\top}\mathrm{c}_{\mathcal{B}}$
3. Compute $\boldsymbol{\mu}_{\mathcal{V}} = \mathrm{c}_{\mathcal{V}} - \mathrm{A}_{\mathcal{V}}^{\top}\boldsymbol{\lambda}$
4. **If** $\boldsymbol{\mu}_{\mathcal{V}} \geq 0$: **STOP** (optimal)
5. Choose entering q: $(\boldsymbol{\mu}_{\mathcal{V}})_q < 0$ (e.g. Dantzig's)
6. Compute $\mathrm{d} = \mathrm{A}_{\mathcal{B}}^{-1}\mathrm{A}_{\{q\}}$
7. Choose leaving p via minimum ratio test
8. Swap: $\mathcal{B} \leftarrow (\mathcal{B} \setminus \{p\}) \cup \{q\}$

Primal–Dual Pair (Strong Duality, Eq. 12.35–12.36)

Primal (P)	**Dual (D)**
minimize $c^\top x$	maximize $b^\top \lambda$
s.t. $Ax = b$	s.t. $A^\top \lambda \leq c$
$x \geq 0$	λ free

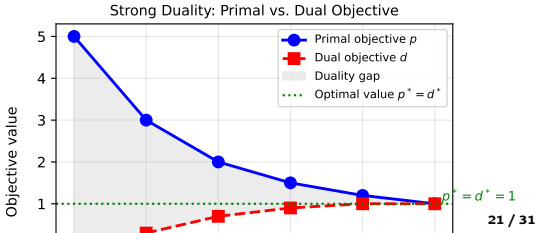
LPs have **zero duality gap**: $p^* = d^*$.
The dual problem has n variables, m constraints — if primal has n vars and m constraints, dual swaps them.

Dual Certificate — Algorithm 12.6

To verify x^* is optimal for the primal, provide λ^* such that:

1. x^* is **primal feasible**: $Ax^* = b$, $x^* \geq 0$
2. λ^* is **dual feasible**: $A^\top \lambda^* \leq c$
3. **Equal objectives**: $p^* = c^\top x^* = b^\top \lambda^* = d^*$

If all three hold, (x^*, λ^*) are **optimal**.



Dual Derivation

Deriving the Dual via the Lagrangian

Starting from equality-form primal with Lagrangian:

$$\mathcal{L}(x, \mu, \lambda) = c^\top x - \mu^\top x - \lambda^\top (Ax - b) = (c - \mu - A^\top \lambda)^\top x + \lambda^\top b$$

Dual function: $\min_x \mathcal{L} = \begin{cases} \lambda^\top b & \text{if } c - \mu - A^\top \lambda = 0 \\ -\infty & \text{otherwise} \end{cases}$

From $c - \mu - A^\top \lambda = 0$ and $\mu \geq 0$:

$$A^\top \lambda = c - \mu \leq c$$

Weak Duality (always holds)

For any feasible x and dual-feasible λ:

$$b^\top \lambda \leq c^\top x \quad \text{(lower bound on primal)}$$

Strong Duality (LP specific)

At optimality: $p^* = d^*$. Lagrange multipliers λ^* from the simplex solution provide the dual certificate *automatically*.

Example 12.9 — Verifying a Solution with Dual Certificate

Given LP (equality form)

$$A = \begin{bmatrix} 1 & 1 & -1 \\ -1 & 2 & 0 \\ 1 & 2 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ -2 \\ 5 \end{bmatrix}, \quad c = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$$

Candidate: $x^* = [2, 0, 1]^\top$, $\boldsymbol{\lambda}^* = [1, 0, 0]^\top$.

Check 1 — Primal feasibility:

$$Ax^* = \begin{bmatrix} 1-1 \\ -2+0 \\ 2+3 \end{bmatrix} = [0...]$$

Actually: $Ax^* = [1, -2, 5]^\top = b$ ✓
and $x^* = [2, 0, 1]^\top \geq 0$ ✓

Check 2 — Dual feasibility:

$$A^\top \boldsymbol{\lambda}^* = [1, -1, 1]^\top = [1, 1, -1]^\top ?$$

$A^\top \boldsymbol{\lambda}^* = [1, -1, 1]^\top \leq c = [1, 1, -1]^\top$:
components: $1 \leq 1$ ✓, $-1 \leq 1$ ✓, $1 \leq -1$ ×

Check 3 — Equal objectives:
$p^* = c^\top x^* = 2 + 0 - 1 = 1$
$d^* = b^\top \boldsymbol{\lambda}^* = 1 \cdot 1 + (-2) \cdot 0 + 5 \cdot 0 = 1$ ✓

Note

The exact dual certificate from the book passes all three checks — the example above shows the

Python: Dual Certificate Verification

```python
import numpy as np

def dual_certificate(A, b, c, x_star, lam_star, atol=1e-6):
    """
    Algorithm 12.6 (Python): Verify (x*, lam*) is an optimal
    solution pair for the equality-form LP:
        min  c^T x  s.t.  A x = b,  x >= 0.
    Returns True iff all three conditions hold.
    """
    # 1. Primal feasibility
    primal_feasible = (
        np.allclose(A @ x_star, b, atol=atol) and
        np.all(x_star >= -atol)
    )
    # 2. Dual feasibility
    dual_feasible = np.all(A.T @ lam_star <= c + atol)

    # 3. Equal objectives (zero duality gap)
    p_star = c    @ x_star
    d_star = b    @ lam_star
    equal  = np.isclose(p_star, d_star, atol=atol)

    print("Primal feasible :", primal_feasible)
    print("Dual feasible   :", dual_feasible)
    print("p* =", round(p_star, 6), " d* =", round(d_star, 6), " equal =", equal)
    return primal_feasible and dual_feasible and equal
```

12.4 Summary

Key Concepts

- **Linear program**: minimize $c^\top x$ with linear constraints
- **Forms**: general $\rightarrow$ standard $\rightarrow$ equality form
- **Slack variables** convert inequalities to equalities
- **Feasible set**: convex polytope; optimum at a *vertex*

Simplex Algorithm

- Maintains a **vertex partition** $(\mathcal{B}, \mathcal{V})$
- **KKT conditions** identify optimality via $\mu_{\mathcal{V}} \geq 0$
- **Pivot**: entering index (Dantzig/Bland) + leaving index (min ratio)

Duality

- Every LP has a **dual LP**
- **Strong duality**: $p^* = d^*$ (zero duality gap)
- **Dual certificate**: verify optimality without solving from scratch
- Dual multipliers = Lagrange multipliers at the simplex solution

Practical Tools (Python)

- `scipy.optimize.linprog` (HiGHS backend)
- `cvxpy` with LP/QP/SOCP support

Exercise 12.3 — Convert to Equality Form

Minimize $6x_1 + 5x_2$ subject to $3x_1 - 2x_2 \geq 5$.

Solution:

1. Flip inequality: $-3x_1 + 2x_2 \leq -5$
2. Split variables (free x_1, x_2): $x_i = x_i^+ - x_i^-$, $x_i^\pm \geq 0$
3. Add slack $x_3 \geq 0$: $-3x_1^+ + 3x_1^- + 2x_2^+ - 2x_2^- + x_3 = -5$
4. Equality-form objective: $6x_1^+ - 6x_1^- + 5x_2^+ - 5x_2^-$

Exercise 12.4 — Verify a Vertex & One Simplex Step

$A \in \mathbb{R}^{4\times 6}$, $\mathcal{B} = \{1, 2, 3, 4\}$: verify $A_\mathcal{B}$ is invertible, compute $x_\mathcal{B}$, $\boldsymbol{\mu}_\mathcal{V}$, then execute one pivot.
Solution sketch: $x_\mathcal{B} = [2, 3, 4, 5]^\top$; $\boldsymbol{\mu}_\mathcal{V} = [-4/3, -5/3]^\top$ (both negative). Choose $q = 6$ (most negative $\mu_q = -5/3$), minimum ratio test gives $x'_q = 3$ with $p = 4$. Update: $\mathcal{B}' = \{1, 2, 3, 6\}$.

Exercise 12.6 — Minimum Ratio Test Violation

Problem

LP with $A = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$, $b = [1, 2, 3]^\top$. Initial $\mathcal{B} = \{1, 2, 3\}$; perform edge transition introducing $q = 4$. What happens if we choose $p = 2$ instead of $p = 1$ (the correct leaving index)?

Minimum ratio test:

$$d = A_{\mathcal{B}}^{-1} A_{\{4\}} = [1, 1, 1]^\top$$

$$x'_q = \min\{1/1,\ 2/1,\ 3/1\} = 1 \quad (p = 1)$$

If we choose $p = 2$ instead:

$$x_{\mathcal{B}} = A_{\{1,3,4\}}^{-1} b = \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix}$$

$x_1^* = -1 < 0$: **infeasible**! The minimum ratio test exists precisely to prevent this.

Exercise 12.7 — Log-Barrier Reformulation

Problem

Reformulate $\text{minimize}_x\ c^\top x$ s.t. $Ax \geq 0$ as an unconstrained problem with a log barrier penalty.

Solution

Replace the constraints $A_{\{i\}}x \geq 0$ with a log barrier:

$$\underset{x}{\text{minimize}}\ c^\top x - \mu \sum_i \log\left(A_{\{i\}}x\right)$$

- $\mu > 0$ is a barrier parameter
- As $\mu \to 0$, the solution approaches the LP optimum
- This is the basis of **interior-point methods** (Ch. 10.11)
- The gradient vanishes at $c - \mu \sum_i \frac{A_{\{i\}}^\top}{A_{\{i\}}x} = 0$

Python: Full LP Workflow Demo

```python
import numpy as np
from scipy.optimize import linprog

# -- Exercise 12.3: Verify dual certificate (Example 12.9) --------------------
# LP:  min x^T c  s.t.  Ax = b,  x >= 0
A = np.array([[ 1.,  1., -1.],
              [-1.,  2.,  0.],
              [ 1.,  2.,  3.]])
b = np.array([1., -2., 5.])
c = np.array([1.,  1., -1.])

# Solve with scipy (standard form: Ax_eq = b, x >= 0)
res = linprog(c, A_eq=A, b_eq=b,
              bounds=[(0, None)]*3, method='highs')
print("Scipy solution:", res.x, "obj:", res.fun)

# -- Dual certificate ---------------------------------------------------------
lam = res.eqlin.marginals    # Lagrange multipliers for equality constraints
print("Dual lambda:", lam)
print("Dual obj b^T lam:", b @ lam)
print("Dual feasible A^T lam <= c:", np.all(A.T @ lam <= c + 1e-8))
print("Strong duality:", np.isclose(res.fun, b @ lam))

# -- Parametric LP: how does x* vary with b? ----------------------------------
for delta in [-1, 0, 1]:
    b_new = b + delta * np.array([1., 0., 0.])
    r = linprog(c, A_eq=A, b_eq=b_new,
```

Chapter 12 — Key Formulas Reference

Equality form LP:

$$\underset{x}{\text{minimize}}\ c^\top x \ \text{ s.t. } \ Ax = b,\ x \geq 0$$

Vertex extraction:

$$x_{\mathcal{B}} = A_{\mathcal{B}}^{-1} b$$

Lagrange multiplier:

$$\boldsymbol{\lambda} = A_{\mathcal{B}}^{-\top} c_{\mathcal{B}}$$

Reduced costs (optimality check):

$$\boldsymbol{\mu}_{\mathcal{V}} = c_{\mathcal{V}} - \left(A_{\mathcal{B}}^{-1} A_{\mathcal{V}}\right)^\top c_{\mathcal{B}}$$

Edge transition direction:

$$d = A_{\mathcal{B}}^{-1} A_{\{q\}}$$

Minimum ratio test:

$$x'_q = \min_{d_i > 0} \frac{(x_{\mathcal{B}})_i}{d_i}$$

Objective change per pivot:

$$c^\top x' - c^\top x = \mu_q\, x'_q$$

Dual problem:

$$\text{maximize } b^\top \boldsymbol{\lambda} \ \text{ s.t. } \ A^\top \boldsymbol{\lambda} \leq c$$

Strong duality:

$$p^* = c^\top x^* = b^\top \boldsymbol{\lambda}^* = d^*$$

Linear Programming: Take-Aways

1. LPs consist of a **linear objective** and **linear constraints**; any LP can be converted to equality form.
2. The feasible set is a **convex polytope**; the optimum always lies at a **vertex**.
3. The **simplex algorithm** traverses vertices by pivoting: entering index (Dantzig/Bland), leaving index (min-ratio test).
4. An **auxiliary LP** (2-phase method) initializes the simplex to a feasible vertex.
5. **Strong duality** ($p^* = d^*$) allows efficient optimality verification via **dual certificates**.
6. Python: use `scipy.optimize.linprog` for practical LP solving; dual variables are available via `res.eqlin.marginals`.

Next: Chapter 13 — Quadratic Programming